```
// routes/auth.js
// Handles account creation and
login.
// Passwords are never stored as
plain text - bcrypt turns them into a
hash.

const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const pool = require('../db/pool');

const router = express.Router();

// POST /api/auth/register
// Creates a new user account.
router.post('/register', async (req,
res) => {
  const { email, password } =
req.body;

  if (!email || !password) {
    return res.status(400).json({
error: 'Email and password are
required.' });
  }
```

```
  try {
    // 10 is the "salt rounds" -
higher = more secure but slower. 10
is a solid default.
    const passwordHash = await
bcrypt.hash(password, 10);

    const result = await pool.query(
      'INSERT INTO users (email,
password_hash) VALUES ($1, $2)
RETURNING id, email',
      [email, passwordHash]
    );

    res.status(201).json({ user:
result.rows[0] });
  } catch (err) {
    if (err.code === '23505') {
      // Postgres error code for
"unique constraint violation" - email
already exists
      return res.status(409).json({
error: 'An account with that email
already exists.' });
    }
    console.error(err);
    res.status(500).json({ error:
'Something went wrong.' });
```

```
  }
});

// POST /api/auth/login
// Checks credentials and returns a
JWT token if valid.
router.post('/login', async (req,
res) => {
  const { email, password } =
req.body;

  try {
    const result = await
pool.query('SELECT * FROM users WHERE
email = $1', [email]);
    const user = result.rows[0];

    if (!user) {
      return res.status(401).json({
error: 'Invalid email or password.'
});
    }

    const passwordMatches = await
bcrypt.compare(password,
user.password_hash);
    if (!passwordMatches) {
      return res.status(401).json({
error: 'Invalid email or password.'
```

```
});
    }

    // Create a token that proves
this user is logged in.
    // The frontend will send this
back on every future request.
    const token = jwt.sign({ userId:
user.id }, process.env.JWT_SECRET, {
      expiresIn: '7d',
    });

    res.json({ token, user: { id:
user.id, email: user.email } });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error:
'Something went wrong.' });
  }
});

module.exports = router;
```